CIM-BLAS: Computing-in-Memory Accelerator for BLAS

Rui Liu*†, Zerun Li*‡, Xiaoyu Zhang*‡, Xiaoming Chen*‡, Yinhe Han*‡, Minghua Tang†§
*Institute of Computing Technology, Chinese Academy of Science
†School of Materials Science and Engineering, Xiangtan University
‡University of Chinese Academy of Sciences
§National Center for Applied Mathematics in Hunan
Corresponding authors: chenxiaoming@ict.ac.cn, tangminghua@xtu.edu.cn

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133288

***Abstract*—Basic Linear Algebra Subprograms (BLAS) is a foundational software library for linear algebra kernels, which is widely used in scientific and engineering computing. Existing BLAS accelerations mainly rely on CPUs and GPUs. Many operations in BLAS are data intensive, so they are constrained by the limited memory bandwidth of CPUs and GPUs. The computing-in-memory (CIM) technology can effectively alleviate the memory wall bottleneck and is particularly suitable for accelerating BLAS. In this paper, we propose the first CIM accelerator for BLAS, CIM-BLAS, based on non-volatile memory. CIM-BLAS includes a unified floating-point pipeline to support high-precision arithmetics. High efficiency of the accelerator is achieved by developing configurable data flows to support various BLAS functions. Compared with GPU implementations, CIM-BLAS demonstrates several orders of magnitude performance and energy efficiency improvements for executing level-1 and level-2 BLAS functions, and can achieve an energy efficiency improvement of 2.6-24.1× for executing level-3 BLAS functions. The improvement increases with the size of the matrix, indicating excellent scalability of CIM-BLAS. Application-level evaluations also demonstrate the potential of CIM for accelerating BLAS.**

***Index Terms*—Computing-in-memory, Non-volatile memory, Basic Linear Algebra Subprograms**

I. Introduction

Linear algebra is the foundation of numerous scientific computing and engineering applications, covering core topics such as matrix operations, vector operations, solving systems of linear equations, eigenvalue problems, etc. Basic Linear Algebra Subprograms (BLAS) is a standardized library for linear algebra computations, which provides a series of common and fundamental linear algebra kernels. Over the past few decades, researchers have made significant efforts to accelerate BLAS on CPU and GPU platforms, continuously optimizing workloads and data communication at the software level [1]–[5]. However, in traditional computing architectures, frequent cross-layer data transfers cause severe transmission energy consumption issues, and the limited memory bandwidth significantly restricts the performance. Especially when handling large-scale matrix operations, the performance bottlenecks of traditional computing architectures become significant.

Computing-in-memory (CIM) breaks through the limitations of traditional architectures by allowing data to be processed in place within memory. This effectively mitigates the memory wall bottleneck by reducing the latency and energy overhead caused by data transfers. Recently, lots of CIM-based accelerators have been proposed, demonstrating better performance and energy efficiency compared with traditional ASIC accelerators. The target applications include neural network training (e.g., [6]–[8]), neural network inference (e.g., [9]–[11]), scientific computing (e.g., [12]–[14]), cryptography (e.g., [15]–[17]), etc. Since most BLAS functions are data intensive, CIM also has great potential, but currently we have not seen studies on CIM-based BLAS accelerators. Different from existing accelerators which typically accelerate a specific class of application, since BLAS provides an interface for common and fundamental linear algebra kernels, accelerating BLAS can benefit a wide range of applications. It faces two major challenges when accelerating BLAS by CIM.

Difficulty in supporting floating-point arithmetics: To enable linear algebra computations, BLAS requires the deployed hardware to support 32- and 64-bit floating-point arithmetics. However, the physical location of data in memory is fixed, and the analog data paths in memory cannot meet the exponent alignment requirements for floating-point arithmetics. Therefore, most existing CIM architectures are limited to performing fixed-point operations [18], [19].

Difficulty in supporting complex functions: BLAS requires more complex computations, but current CIM architectures only support a limited set of fixed operations [20]–[22]. Complex calculations such as matrix transposition, subtraction, and division often require other co-processors or additional peripheral modules, which inevitably cause significant communication or area overhead.

In this work, we for the first time attempt deploying the standard library BLAS for fundamental vector and matrix kernels onto CIM hardware, and propose a CIM-based BLAS accelerator, CIM-BLAS. The main contributions of this paper

This work was supported by National Key R&D Program of China (No. 2023YFF0719600), by National Natural Science Foundation of China (Nos. 62488101, 62495104, 62122076 and U23A20322), by Youth Innovation Promotion Association CAS, by Hunan Provincial Natural Science Foundation (Grant Nos. 2023JJ50009 and 2023JJ30599), by the Foundation of Innovation Center of Radiation Application (No. KFZC2023020701), and by Postgraduate Scientific Research Innovation Project of Xiangtan University (No. XDCX2023Y184).

are as follows.

- We study the design challenges of deploying BLAS on CIM-based hardware, and propose the first computation-in-memory BLAS accelerator based on non-volatile memory, which includes a unified floating-point pipeline to support high-precision arithmetics.
- We map complex arithmetic operations onto CIM arrays with conventional peripheral modules, and design a configurable data flow for different BLAS functions, enabling efficient BLAS acceleration without the need for additional hardware modules.
- A comprehensive evaluation of the CIM-BLAS architecture is conducted. Performance benchmarking indicates significant advantages over GPU implementations for executing level-1 and level-2 BLAS functions. For executing level-3 BLAS functions, the energy efficiency improves by 2.6-24.1× and increases with the size of the matrix. Application-level evaluations also demonstrate the potential of CIM for accelerating BLAS.

II. Motivation

BLAS is a typical data-intensive application, involving computations on matrices and vectors, such as matrix multiplication and vector addition. BLAS functions include three levels: level-1 for vector-vector operations [23], level-2 for matrix-vector operations [24], and level-3 for matrix-matrix operations [25], [26]. These operations require extensive memory access and data transfer, especially when handling large-scale matrices, making them well-suited for acceleration with CIM architectures. However, existing CIM architectures have certain limitations in supporting floating-point operations and complex linear algebra operations, such as matrix transposition, subtraction, and division.

This work aims to explore the potential of CIM architectures in supporting complex linear algebra operations. We observe that the primary difficulty in supporting floating-point operations with CIM is the exponent alignment problem, and it can be achieved using a delayed mantissa input method, which equivalently transforms the original floating-point operations into fixed-point operations within CIM arrays. We further design a new data flow and execution mechanisms to extend the potential of CIM arrays with peripheral modules, to support various BLAS functions. We hope to overcome the limitations of existing CIM architectures while enhancing the efficiency of BLAS computations.

III. CIM-BLAS Architecture Design

A. Architecture Overview and Data Layout

Fig. 1 shows the proposed CIM-BLAS accelerator based on the IEEE-754 floating-point format [27]. Processing engines (PEs)/computation units (CUs) are interconnected with an H-tree topology. In the H-tree topology, the nodes serve as computing units that can aggregate intermediate results from CUs, thereby allowing flexible configuration of the CUs to efficiently handle various BLAS functions.

A CU integrates multiple crossbar arrays which are dedicated to performing the basic operations of BLAS, which

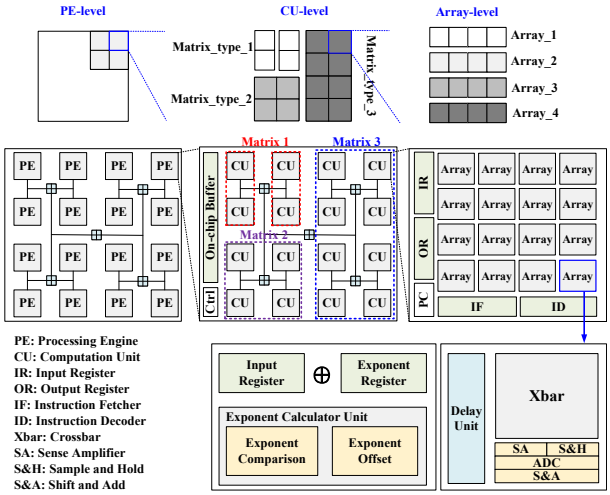

Fig. 1: CIM-BLAS accelerator overview.

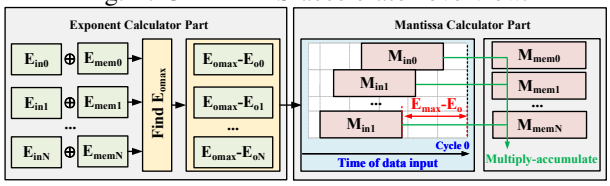

Fig. 2: In-memory floating-point operation pipeline.

can be implemented with popular non-volatile devices (e.g., RRAM, PCM, FeFET, MRAM). Each array contains a set of input registers and exponent registers, an exponent computation unit, a delay unit, and a crossbar equipped with a set of peripheral components. The floating-point data stored in the array is split into exponent and mantissa, which are stored in the exponent registers and the crossbar, respectively. It is worth noting that by adding a bias constant to the mantissas to represent negative numbers, the sign bit can be ignored [9]. The input registers are used to receive data to be processed and split it into exponent and mantissa. The exponent calculation unit, delay unit, and crossbar are used to implement the multiply-accumulate operations for floating-point numbers.

Fig. 1 also illustrates the data layout in the accelerator. A matrix is directly mapped onto different PEs on the chip. The sub-matrix in each PE is further partitioned into blocks and stored in different CUs. During computation, these blocks are combined through the nodes of the H-tree. In a CU, the data is split horizontally, with each array storing a row of the matrix. Different matrices can also be mapped within the same PE/CU to meet various matrix computation requirements.

B. In-Memory Floating-Point Pipeline

Non-volatile memory-based crossbars are widely used for accelerating neural networks. However, they are mostly designed for low-precision fixed-point computations which cannot meet the high-precision requirement of BLAS. Based on the floating-point storage in Section III-A, we propose a unified 5-stage pipeline mechanism for in-memory floating-point operations, which is shown in Fig. 2.

In the 1st stage, the exponent stored in the input register is added to the corresponding data in the exponent register. The exponent output results (E_o) of all rows are sent to the exponent comparison unit in the 2nd stage to find their

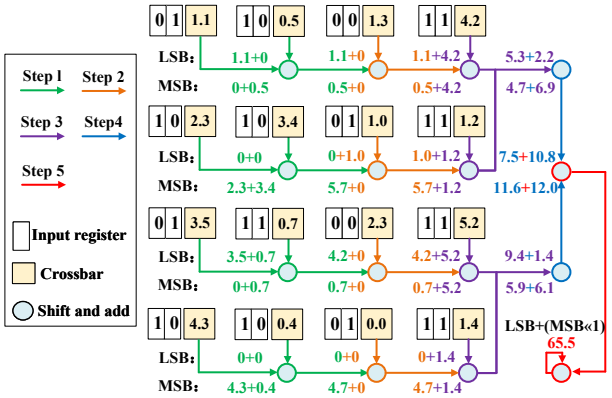

Fig. 3: Data flow of the Ripple pipeline.

maximum value (E_{omax}). In the 3rd stage, the exponent offset for each row is calculated by $E_{omax} - E_o$ with the exponent comparison units. In the 4th stage, according to the obtained exponent offsets, the corresponding input mantissas are shifted and some bits '0' are padded in the lower bits, thereby achieving exponent alignment. In other words, the delay units construct a new input, transforming the original floating-point operation into an equivalent fixed-point operation. In the 5th stage, the crossbar array receives the shifted mantissa data from the delay unit and performs in-memory integer multiply-accumulate operations. Finally, the floating-point results can be obtained by combining the exponent and mantissas parts. Notably, when only one multiplication operation is performed in the array, no alignment operation is needed, and only the 1st and 5th stages are executed. In this case, exponents and mantissas can be processed at the same time.

In the remaining part of this section, we will show in detail the data flow of each function of BLAS based on the CIM-BLAS architecture with the unified floating-point pipeline. Various BLAS functions can be efficiently accelerated on the unified hardware architecture by configuring the data path.

C. SCAL, DOT, ASUM, and NRM2 Functions

The SCAL, ASUM, and DOT functions are part of level-1 BLAS, which are used for vector-scalar multiplication, sum, and dot product operations, respectively.

For the SCAL function, as described in Section III-A, the matrix is mapped to the crossbar arrays in a row-wise manner. Note that a vector can be regarded as a matrix with N rows and 1 column, which means that each element of the vector is distributed across different arrays. Therefore, each array will perform a floating-point multiplication operation in parallel with the data sent to their input registers. These computation results will be collected by the CUs' output registers or the PEs' on-chip buffer for subsequent operations.

For the DOT function, the products of corresponding elements from two vectors need to be accumulated. However, traditional architectures rely on an addition tree to sum the output results from the array, which incurs additional area and latency overhead. To maximize the potential of the peripheral components and to avoid introducing additional hardware, we propose a pipeline structure to efficiently support dot-product operations, named Ripple. Fig. 3 illustrates the bit-serial execution flow of the Ripple pipeline. Here, we assume that each CU is configured with 4×4 arrays, with each array's input register storing a 4-bit mantissa. The green, orange, purple, blue, and red nodes are shift-and-add units of the arrays. By using the shifters for alignment operations, they can be extended to perform floating-point addition. We construct a systolic-like structure with those existing components. Each node will receive data from the array or the previous node and perform an accumulation operation. The data flows along the row direction, and the partial sum of the dot product operation can be obtained at the blue node. By accumulating these partial sums through the red node, the final result can be obtained.

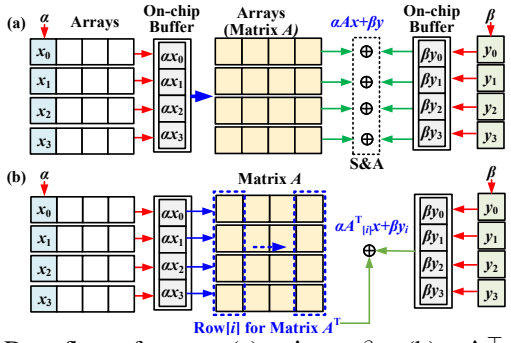

Fig. 4: Data flow of GEMV. (a) $\alpha\mathbf{A}\mathbf{x} + \beta\mathbf{y}$. (b) $\alpha\mathbf{A}^\top\mathbf{x} + \beta\mathbf{y}$.

ASUM and NRM2 can be considered as sub-operations of the DOT function. The ASUM function computes the sum of the elements in a vector. When performing an ASUM function, we only need to read out the corresponding data from each array and use the Ripple pipeline to execute the sum operation.

The basic formula of NRM2 function is $nrm2(\mathbf{x}) = \sqrt{\sum_{i=1}^{n} x_i^2}$, where $\mathbf{x}$ is a vector with elements x_i's, and n is the number of elements in the vector. To compute NRM2, each array first performs a read operation and feeds the read data back into its input register. Finally, the vector performs a DOT function with itself to obtain an intermediate result ($\sum_{i=1}^{n} x_i^2$), which is then sent to a co-processor or CPU for square root calculation. It is worth noting that the amount of data required to be transmitted to the co-processor or CPU for the NRM2 function is very small, which is only 64-bit data for a double-precision floating-point. Therefore, the communication overhead is negligible compared with the computation involved.

D. GEMV and GEMM Functions

GEMV is an important function in level-2 BLAS, which is used for matrix-vector multiplication. Specifically, GEMV performs the following operations

$$\mathbf{y} \leftarrow \alpha\mathbf{A}\mathbf{x} + \beta\mathbf{y}, \text{ or } \mathbf{y} \leftarrow \alpha\mathbf{A}^\top\mathbf{x} + \beta\mathbf{y} \quad (1)$$

where $\mathbf{A}$ is an $m \times n$ matrix, x is an n-dimensional vector, y is an m-dimensional vector, and α and β are scalars.

Fig. 4 illustrates the data flow of the GEMV function. In order to simplify the computation process, vectors $\mathbf{x}$ and $\mathbf{y}$ first perform SCAL functions to obtain $\alpha\mathbf{x}$ and $\beta\mathbf{y}$ at the same time. Then the vector $\alpha\mathbf{x}$ is loaded into the input registers of the CUs where matrix $\mathbf{A}$ resides and subsequently sent to the corresponding arrays to perform multiply-accumulate

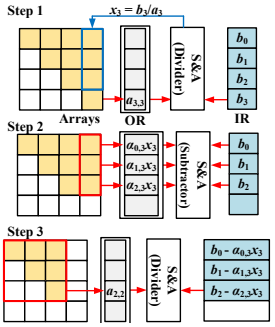

Fig. 5: Data flow of solving an upper triangular matrix.

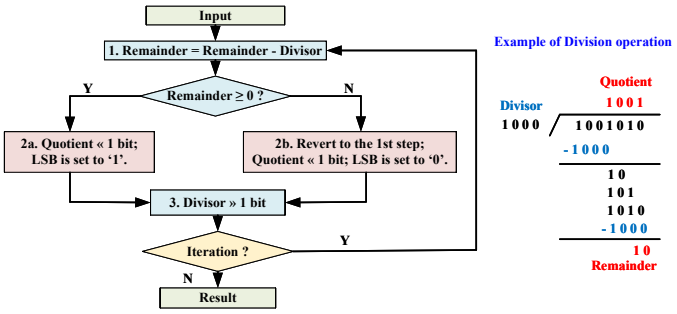

Fig. 6: Division operation implementation with shift-and-add module.

operations in parallel. The elements of the output column vector $\alpha\mathbf{Ax}$ are the output results of the arrays. Finally, $\beta\mathbf{y}$ is added to the output column vector $\alpha\mathbf{Ax}$ with shift-and-add modules of each array, obtaining the result of the GEMV function.

When matrix $\mathbf{A}$ needs an additional transpose operation, each array can be considered as storing a column vector of matrix $\mathbf{A}^\top$. Due to the physical storage location of the data, the elements of the same row vector of $\mathbf{A}^\top$ are dispersed across various arrays. This prevents us from relying on the arrays to perform matrix-vector multiplication operations in parallel to compute the output column vector. In this case, we can utilize the Ripple pipeline to perform the computation of $\alpha\mathbf{A}^\top\mathbf{x}$. We divide the matrix $\mathbf{A}^\top$ into N row vectors and perform DOT functions with vector $\alpha\mathbf{x}$ sequentially, which allows us to obtain the elements of the output column vector one by one. Finally, the elements of the output column vector will be added to the corresponding elements of vector $\beta\mathbf{y}$ to obtain the result of $\alpha\mathbf{A}^\top\mathbf{x} + \beta\mathbf{y}$.

In our accelerator, GEMM is implemented based on GEMV. We split the second input matrix into multiple column vectors and sequentially execute the GEMV function with the target matrix until all column vectors have been processed.

E. TRSV *Function*

The TRSV function is one of the most widely used kernel in BLAS, which solvers triangular matrix problems. Its operation can be defined by the following formula

$$\mathbf{Ax} = \mathbf{b} \tag{2}$$

where $\mathbf{A}$ is an $n \times n$ triangular matrix, $\mathbf{x}$ represents the unknown vector to be solved, and $\mathbf{b}$ is a constant vector.

We use forward/backward substitution to solve upper/lower triangular matrices. Fig. 5 shows an example of solving an upper triangular matrix. In the first step, the element $a_{3,3}$ is read out and a division operation is performed with by b_3. At this point, we obtain the solution x_3 for the first equation. In the second step, x_3 is fed back into the matrix $\mathbf{A}$, and SCAL functions are performed. The constant term vector b_i is subtracted by the result of $a_{i,3}x_3$ for the corresponding row, and the result is stored in the registers. The remaining elements that are not involved in the computation will form a new upper triangular matrix. We repeat the above steps until all unknown variables x_i are solved.

To avoid hardware overhead from additional peripheral components, we also use the shift-and-add module to support

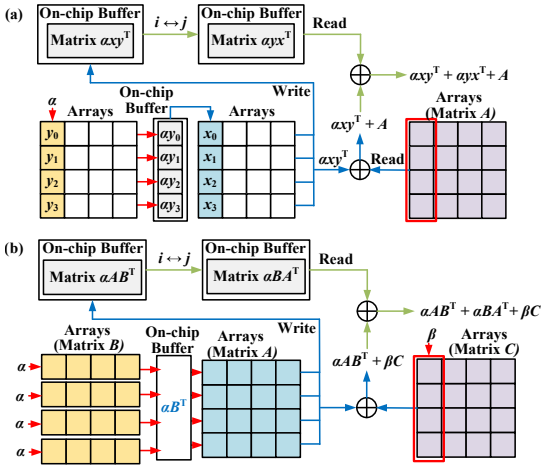

Fig. 7: (a) Data flow of GER, SYR, and SYR2 functions. (b) Data flow of SYR2k function.

matrix subtraction and division operations. To obtain $\mathbf{A} - \mathbf{B}$, we keep the minuend $\mathbf{A}$ unchanged and take the two's complement of the subtrahend $\mathbf{B}$. In this way, $\mathbf{A} - \mathbf{B}$ will be converted into an add operation of $\mathbf{A}$ and the two's complement of $\mathbf{B}$. The division operation implementation primarily rely on the subtraction and shift operations, as shown in Fig. 6. First, we calculate the difference between the remainder and the divisor, and check if the result is non-negative. If the value is negative, it reverts to the 1st step, left-shifts the quotient by 1 bit, and sets the new rightmost bit to '0'; otherwise, we left-shift the quotient by 1 bit, and set the new rightmost bit to '1'. The divisor register is right-shifted by 1 bit. The divisor is right-shifted by 1 bit and checked whether all the computational operations have been completed. If not, we return to the 1st step and continue executing the above process.

F. GER, SYR, SYR2, SYRK, *and* SYR2K *Functions*

GER, SYR, and SYR2 are functions widely executed in data analysis, optimization problems, and other scientific computing applications, which can effectively handle incremental updates to a matrix. Their specific operations are as follows

$$\mathbf{A} \leftarrow \alpha\mathbf{xy}^\top + \mathbf{A}, \quad \mathbf{A} \leftarrow \alpha\mathbf{xy}^\top + \alpha\mathbf{yx}^\top + \mathbf{A}. \tag{3}$$

Fig. 7a provides a detailed description of the data flow of GER, SYR, and SYR2 functions. Multiplying a column vector $\mathbf{x}$ by a row vector $\mathbf{y}^\top$ results in a product matrix $\mathbf{xy}^\top$. To enhance computational parallelism, this product matrix can be transformed into a combination of scalar multiplications

of the column vector elements. To perform a SYR function, vector $\mathbf{y}$ first performs a SCAL function with scalar α, and the result is then sequentially sent to input registers of the CUs where vector $\mathbf{x}$ resides. In this way, we can obtain the result of $\alpha\mathbf{x}\mathbf{y}^\top$ column by column. Each calculation result is sent to the corresponding column of matrix $\mathbf{A}$ for addition with shift-and-add modules. When all elements of vector $\alpha\mathbf{y}$ have been processed, we obtain the SYR function result.

For SYR2, as shown in Eq. (3), the SYR function is a sub-operation of the SYR2 function. Furthermore, $\alpha\mathbf{x}\mathbf{y}^\top$ is the transposes of $\alpha\mathbf{y}\mathbf{x}^\top$. To eliminate redundant computations, we store the results of $\alpha\mathbf{x}\mathbf{y}^\top$ in the buffer and swap their row and column indices. Finally, adding $\alpha\mathbf{y}\mathbf{x}^\top$ from the buffer to the corresponding elements of the SYR function matrix will yield the final result of the SYR2 function.

SYRK and SYR2K are described by the following formulas:

$$\mathbf{A} \leftarrow \alpha\mathbf{A}\mathbf{A}^\top + \beta\mathbf{C},\ \mathbf{A} \leftarrow \alpha\mathbf{A}\mathbf{B}^\top + \alpha\mathbf{B}\mathbf{A}^\top + \beta\mathbf{C}. \quad (4)$$

For the SYRK function, we first apply the SCAL function to all column vectors of matrices $\mathbf{A}$ and $\mathbf{C}$, obtaining $\alpha\mathbf{A}$ and $\alpha\mathbf{C}$, respectively. Afterward, the result of SYRK can be obtained by adding the result of performing GEMM with matrix $\mathbf{A}$ and $\alpha\mathbf{A}^\top$ to the corresponding elements of $\beta\mathbf{C}$. Fig. 7b shows the execution flow of SYR2K function. To obtain the result of the SYR2K function, we first compute the results for $\alpha\mathbf{B}^\top$ and $\beta\mathbf{C}$. Similar to the SYR2 function, we obtain the column vectors of $\alpha\mathbf{A}\mathbf{B}^\top$ sequentially using the GEMV function, and while adding them to the corresponding elements of $\beta\mathbf{C}$, $\alpha\mathbf{A}\mathbf{B}^\top$ is also written into the buffer. By swapping the indices in the buffer, we can easily obtain the result for matrix $\alpha\mathbf{B}\mathbf{A}^\top$. Finally, adding the elements of $\alpha\mathbf{B}\mathbf{A}^\top$ in the buffer to the corresponding elements of matrix storing the intermediate results yields the final result of the SYR2K function.

G. Summary

Based on the unified hardware architecture and floating-point execution pipeline, we have constructed a comprehensive data flow for each BLAS function, covering all BLAS functions except ROTG and AMAX, which require more specialized modules and designs. A Ripple pipeline structure is proposed to efficiently support the SCAL, DOT, ASUM, NRM2 functions, and matrix transpose operation. An in-memory floating-point computation mechanism is used to support the GEMV and GEMM functions. We further map the subtraction and division functions to the shift-and-add module, enabling in-memory TRSV and TRSM functions. Finally, the GER, SYR, SYR2, SYRK, and SYR2K functions are deployed on our accelerator with parallel computation flows.

IV. Evaluation

The CIM arrays and all analog components are simulated with HSPICE. We use the 45nm Predictive Technology Model (PTM) [28] as the base MOSFET model and select RRAMs with a resistance range of 5kΩ-5MΩ [29], [30] to construct the memory arrays. Synopsys Design Compiler is used for synthesizing the digital components. The size of the CIM arrays is 256×256, with 16 shared ADCs used in each array. For the ADCs and DACs used in the accelerator, we obtain their parameters from existing designs/products [31], [32]. Based on these fundamental parameters, we evaluate the performance and energy efficiency of various functions on CIM-BLAS and compare CIM-BLAS with the cuBLAS library that runs on an NVIDIA Tesla V100 GPU (fabricated with 12nm technology). It is worth emphasizing that currently there is no work on CIM- or ASIC-based accelerators for BLAS, so no such baseline can be compared.

A. Comparison with Level-1 and Level-2 cuBLAS

Fig. 8 shows a comparison of performance and energy efficiency between CIM-BLAS and GPU for the DDOT, DGEMV, DTRSV, DSYR, and DSYR2 functions.

For the DDOT and DGEMV functions, the GPU's execution time varies with the size of the matrix/vector. However, for CIM-BLAS, each array can perform matrix multiplication operations simultaneously, and the Ripple pipeline execution cycle of DOT only depends on the array size, so the time for aggregating data is very short. As a result, the execution time of our accelerator remains almost unchanged when performing DDOT and DGEMV functions. Compared with the GPU, CIM-BLAS performs the DDOT and DGEMV functions with speedups of 4290–26108× and 2503–247774×, respectively, while reducing energy consumption by 4101–5391× and 1829–2828×, respectively.

For the DTRSV function, CIM-BLAS demonstrates speedups of 219-3103× and energy savings of 5248-50423× over the GPU baseline. The main reason for the substantial energy savings is that the array relies solely on extremely low-power precharge amplifiers for row-wise calculations. Additionally, the significant communication energy consumption of the GPU further amplifies the disparity.

For the DSYR and DSYR2 functions, CIM-BLAS achieves energy savings of 5323-6471× and a maximum speedup of 12.4× . The main reason is the use of a column-wise computation mechanism, where the execution time is proportional to the number of elements in vector $\mathbf{y}$. This serial computation approach results in longer computation times.

B. Comparison with Level-3 cuBLAS

We compare the performance and energy consumption of executing matrix-matrix multiplication operations, as shown in Fig. 9. GEMM_T denotes the execution of a transposition operation on matrix $\mathbf{A}$, achieved through the Ripple pipeline.

In terms of energy consumption, as the array size increases, CIM-BLAS's energy for the DGEMM, DGEMM_T, DSYRK, and DSYR2K functions continually rises, eventually exceeding the energy consumption of the GPU. This is because CIM-BLAS drives the entire array to perform computations, and the energy consumption of in-memory floating-point multiply operations is inherently higher than that of GPU cores. Therefore, as the matrix size increases, the computation workload increases, the proportion of memory access energy consumption decreases, and the energy consumption gap between CIM-BLAS and GPU cores accumulates and gradually widens, eventually exceeding the GPU's memory communication energy consumption. Benefiting from the low-power precharge circuitry,

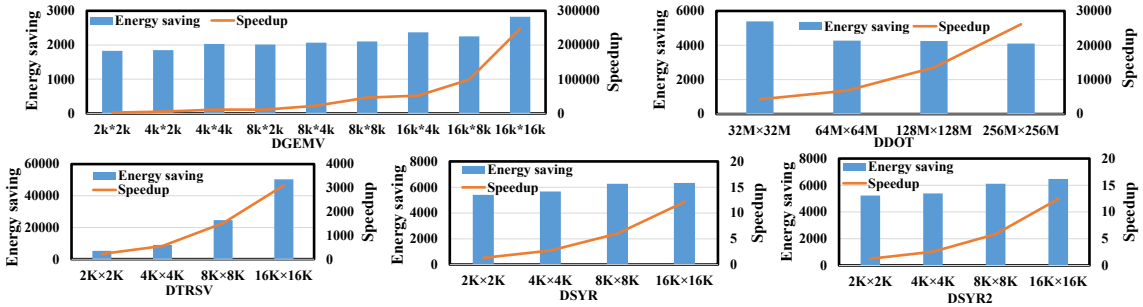

Fig. 8: Speedups and energy savings of CIM-BLAS against GPU for level-1 and level-2 cuBLAS.

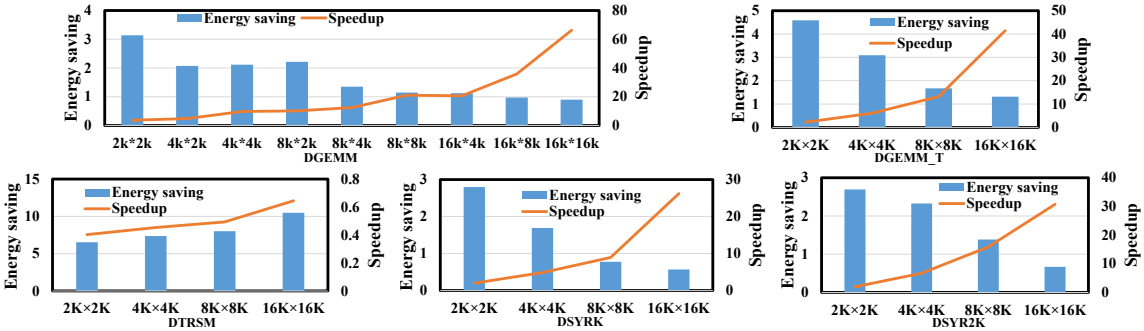

Fig. 9: Speedups and energy savings of CIM-BLAS against GPU for level-3 cuBLAS.

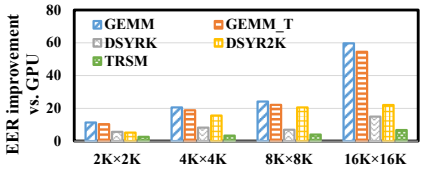

Fig. 10: Energy efficiency ratio (EER) improvements of CIM-BLAS against GPU.

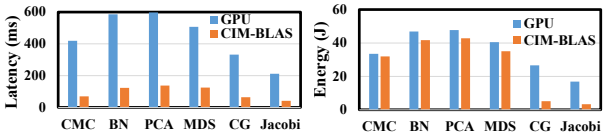

Fig. 11: Application-level latency and energy comparison for different applications.

CIM-BLAS's energy consumption in the DTRSM function is significantly lower than that of the GPU cores, demonstrating energy savings of 6.5-10.5× compared with the GPU baseline.

In terms of performance, for the DGEMM, DGEMM_T, DSYRK, and DSYR2K functions, CIM-BLAS exhibits 1.9-66.3× speedup compared with the GPU. The speedup is even higher for larger matrix sizes. The performance improvement primarily comes from the reduced inter-layer communication and high parallelism of CIM-BLAS. At the current scale, there is a performance gap between DTRSM and the GPU. The main reason is that we use forward/backward substitution to solve upper/lower triangular matrices, which results in longer solving times due to this serial computation approach. However, as the size scale increases, CIM-BLAS shows an upward trend in acceleration. It is expected that CIM-BLAS will surpass the GPU in efficiency for executing the DTRSM function with larger matrix sizes.

Fig. 10 shows the energy efficiency ratio (EER) improvements of CIM-BLAS against the GPU baseline. For various functions, CIM-BLAS's EER is 2.6–24.1× higher than that of the GPU. The EER advantage of CIM-BLAS increases with the matrix size, showing an upward trend as the size grows. This also confirms that the proposed CIM-BLAS can efficiently support BLAS functions.

C. Application-Level Evaluation

We use 6 applications related to scientific computing for the application-level evaluation: covariance matrix calculation (CMC), Bayesian network (BN), principal component analysis (PCA), multidimensional scaling (MDS), conjugate gradient (CG), and Jacobi iterative method. As shown in Fig. 11, CIM-BLAS reduces the execution latency by 4.88× on average. Due to the dominant matrix-matrix operations, CIM-BLAS have limited energy savings in CMC, BN, PCA, and MDS. CIM-BLAS's efficient matrix-vector operations reduce the energy consumption of CG and Jacobi by 5.15× on average.

V. Conclusion

BLAS is a fundamental software library for linear algebra kernels. It is typically implemented on CPUs or GPUs, whose performance is heavily restricted by the memory bandwidth on conventional von Neumann computers. Due to the in-situ computation capability, CIM is evidently a promising solution for enhancing BLAS performance. This paper presents the first CIM-based BLAS accelerator architecture, which includes a unified floating-point pipeline to support high-precision arithmetics. By developing the potential of CIM arrays and peripheral components, we further propose a configurable data flow for various BLAS functions, achieving high efficiency for BLAS executions. Evaluation results show that, compared with GPU implementations, our CIM accelerator has significant potential for accelerating BLAS.

REFERENCES

[1] L. Wang, W. Wu, Z. Xu, J. Xiao, and Y. Yang, "Blasx: A high performance level-3 blas library for heterogeneous multi-gpu computing," in *Proceedings of the 2016 International Conference on Supercomputing*, 2016, pp. 1–11.

[2] A. Abdelfattah, D. Keyes, and H. Ltaief, "Kblas: An optimized library for dense matrix-vector multiplication on gpu accelerators," *ACM Transactions on Mathematical Software (TOMS)*, vol. 42, no. 3, pp. 1–31, 2016.

[3] cublas: Basic linear algebra on nvidia gpus. [Online]. Available: https://developer.nvidia.com/cublas (Access Date: January 2023)

[4] Amd, amd core math library (acml). [Online]. Available: http://developer.amd.com/acml

[5] J. Dongarra, M. Gates, A. Haidar, J. Kurzak, P. Luszczek, S. Tomov, and I. Yamazaki, "Accelerating numerical dense linear algebra calculations with gpus," *Numerical computations with GPUs*, pp. 3–28, 2014.

[6] H. Jiang, X. Peng, S. Huang, and S. Yu, "Cimat: A transpose sram-based compute-in-memory architecture for deep neural network on-chip training," in *Proceedings of the International Symposium on Memory Systems*, 2019, pp. 490–496.

[7] Z. Wang, C. Li, P. Lin, M. Rao, Y. Nie, W. Song, Q. Qiu, Y. Li, P. Yan, J. P. Strachan, N. Ge, N. McDonald, Q. Wu, M. Hu, H. Wu, R. S. Williams, Q. Xia, and J. J. Yang, "In situ training of feed-forward and recurrent convolutional memristor networks," *Nature Machine Intelligence*, vol. 1, no. 9, pp. 434–442, 2019.

[8] Y. Luo and S. Yu, "Accelerating deep neural network in-situ training with non-volatile and volatile memory based hybrid precision synapses," *IEEE Transactions on Computers*, vol. 69, no. 8, pp. 1113–1127, 2020.

[9] A. Shafiee, A. Nag, N. Muralimanohar, R. Balasubramonian, J. P. Strachan, M. Hu, R. S. Williams, and V. Srikumar, "Isaac: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars," in *2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 14–26.

[10] T. Song, X. Chen, X. Zhang, and Y. Han, "Brahms: Beyond conventional rram-based neural network accelerators using hybrid analog memory system," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 1033–1038.

[11] X. Zhang, R. Liu, T. Song, Y. Yang, Y. Han, and X. Chen, "Re-femat: A reconfigurable multifunctional fefet-based memory architecture," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 11, pp. 5071–5084, 2022.

[12] X. Chen and Y. Han, "Solving least-squares fitting in $o(1)$ using rram-based computing-in-memory technique," in *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2022, pp. 339–344.

[13] X. Yin, Y. Qian, A. Vardar, M. Günther, F. Müller, N. Laleni, Z. Zhao, Z. Jiang, Z. Shi, Y. Shi, X. Gong, C. Zhuo, T. Kämpfe, and K. Ni, "Ferroelectric compute-in-memory annealer for combinatorial optimization problems," *Nature Communications*, vol. 15, no. 1, p. 2419, 2024.

[14] S. Wang, Z. Sun, Y. Liu, S. Bao, Y. Cai, D. Ielmini, and R. Huang, "Optimization schemes for in-memory linear regression circuit with memristor arrays," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 68, no. 12, pp. 4900–4909, 2021.

[15] M. Xie, S. Li, A. O. Glova, J. Hu, and Y. Xie, "Securing emerging nonvolatile main memory with fast and energy-efficient aes in-memory implementation," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 26, no. 11, pp. 2443–2455, 2018.

[16] M. Ma, Y. Zhu, Z. Zhu, R. Yuan, J. Liu, L. Xu, Y. Yang, and Y. Wang, "Efficient in-memory aes encryption implementation using a general memristive logic: Surmounting the data movement bottleneck," *IEEE Nanotechnology Magazine*, vol. 16, no. 2, pp. 24–C3, 2022.

[17] R. Liu, X. Zhang, Z. Xie, X. Wang, Z. Li, X. Chen, Y. Han, and M. Tang, "Fecrypto: instruction set architecture for cryptographic algorithms based on fefet-based in-memory computing," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 9, pp. 2889–2902, 2023.

[18] P. Chi, S. Li, C. Xu, T. Zhang, J. Zhao, Y. Liu, Y. Wang, and Y. Xie, "Prime: A novel processing-in-memory architecture for neural network computation in reram-based main memory," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 27–39, 2016.

[19] W. Wan, R. Kubendran, C. Schaefer, S. B. Eryilmaz, W. Zhang, D. Wu, S. Deiss, P. Raina, H. Qian, B. Gao, S. Joshi, H. Wu, H.-S. P. Wong, and G. Cauwenberghs, "A compute-in-memory chip based on resistive random-access memory," *Nature*, vol. 608, no. 7923, pp. 504–512, 2022.

[20] A. Agrawal, A. Jaiswal, C. Lee, and K. Roy, "X-sram: Enabling in-memory boolean computations in cmos static random access memories," *TCAS-I*, vol. 65, no. 12, pp. 4219–4232, 2018.

[21] X. Chen, Y. Wu, and Y. Han, "Fepim: Contention-free in-memory computing based on ferroelectric field-effect transistors," in *Proceedings of the 26th Asia and South Pacific Design Automation Conference*, 2021, pp. 114–119.

[22] R. Liu, X. Zhang, X. Chen, Y. Han, and M. Tang, "Femic: Multi-operands in-memory computing based on fefets," in *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2022, pp. 678–683.

[23] C. L. Lawson, R. J. Hanson, D. R. Kincaid, and F. T. Krogh, "Basic linear algebra subprograms for fortran usage," *ACM Transactions on Mathematical Software (TOMS)*, vol. 5, no. 3, pp. 308–323, 1979.

[24] J. J. Dongarra, J. Du Croz, S. Hammarling, and R. J. Hanson, "An extended set of fortran basic linear algebra subprograms," *ACM Trans. Math. Softw.*, vol. 14, no. 1, p. 1–17, 1988.

[25] J. J. Dongarra, J. D. Cruz, S. Hammarling, and I. S. Duff, "Algorithm 679: A set of level 3 basic linear algebra subprograms: model implementation and test programs," vol. 16, no. 1, p. 18–28, 1990.

[26] J. J. Dongarra, J. Du Croz, S. Hammarling, and I. S. Duff, "A set of level 3 basic linear algebra subprograms," vol. 16, no. 1, p. 1–17, 1990.

[27] W. Kahan, "Ieee standard 754 for binary floating-point arithmetic," *Lecture Notes on the Status of IEEE*, vol. 754, no. 94720-1776, p. 11, 1996.

[28] W. Zhao and Y. Cao, "New generation of predictive technology model for sub-45 nm early design exploration," *IEEE Transactions on electron Devices*, vol. 53, no. 11, pp. 2816–2823, 2006.

[29] H.-L. Chang, H.-C. Li, C. Liu, F. Chen, and M.-J. Tsai, "Physical mechanism of hfo 2-based bipolar resistive random access memory," in *Proceedings of 2011 International Symposium on VLSI Technology, Systems and Applications*. IEEE, 2011, pp. 1–2.

[30] S.-S. Sheu, P.-C. Chiang, W.-P. Lin, H.-Y. Lee, P.-S. Chen, Y.-S. Chen, T.-Y. Wu, F. T. Chen, K.-L. Su, M.-J. Kao, K.-H. Cheng, and M.-J. Tsai, "A 5ns fast write multi-level non-volatile 1 k bits rram memory with advance write scheme," in *2009 Symposium on VLSI Circuits*. IEEE, 2009, pp. 82–83.

[31] L. Kull, T. Toifl, M. Schmatz, P. A. Francese, C. Menolfi, M. Braendli, M. Kossel, T. Morf, T. M. Andersen, and Y. Leblebici, "A 3.1 mw 8b 1.2 gs/s single-channel asynchronous sar adc with alternate comparators for enhanced speed in 32 nm digital soi cmos," *IEEE Journal of Solid-State Circuits*, vol. 48, no. 12, pp. 3049–3058, 2013.

[32] T.-H. Yang, H.-Y. Cheng, C.-L. Yang, I.-C. Tseng, H.-W. Hu, H.-S. Chang, and H.-P. Li, "Sparse reram engine: Joint exploration of activation and weight sparsity in compressed neural networks," in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019, pp. 236–249.

Re^{4}PUF: A Reliable, Reconfigurable ReRAM-based PUF Resilient to DNN and Side Channel Attacks

Ning Lin[1,2,3,4,5,*], Yi Li[1,*], Yangu He[1,*], Songqi Wang[1], Hegan Chen[1,5], Kwunhang Wong[1,5], Chuxin Li[1], Jichang Yang[1], Yifei Yu[1], Meng Xu[1], Yongkang Han[7], Rui Chen[2], Xiaoming Chen[6], Xiaoxin Xu[2,3], Jianguo Yang[2,3], Dashan Shang[2,3,†], Zhongrui Wang[4,5,†]
[1]Department of Electrical and Electronic Engineering, The University of Hong Kong, Hong Kong, China; [2]State Key Lab of Fabrication Technologies for Integrated Circuits, Institute of Microelectronics of the Chinese Academy of Sciences (CAS), Beijing, China; [3]Key Laboratory of Microelectronic Devices and Integrated Technology, Institute of Microelectronics of CAS, Beijing, China; [4]School of Microelectronics, Southern University of Science and Technology, Shenzhen, China; [5]AI Chip Center for Emerging Smart Systems (ACCESS), Hong Kong, China; [6]Institute of Computing Technology of CAS; [7]Zhangjiang Lab, Shanghai, China.
*Equal contribution to this work. †Corresponding author: wangzr@sustech.edu.cn;shangdashan@ime.ac.cn.

2025 62nd ACM/IEEE Design Automation Conference (DAC) | 979-8-3315-0304-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/DAC63849.2025.11133290

***Abstract*—Resistive random-access memory (ReRAM) based Physical Unclonable Functions (PUFs) have emerged as an attractive hardware security primitive due to their low energy consumption and compact footprint. However, the reliability of existing ReRAM-based PUFs is challenged by read noise and temperature variations, as well as their resistance to Deep Neural Network (DNN) modeling attacks and Side Channel Attacks (SCAs). In this paper, we propose a novel 3T2R ReRAM-based reconfigurable PUF to address these challenges. By adopting the digital 3T2R voltage division cell design, we improve its reliability against ReRAM read noise and temperature variations, while the adjustable analog supply voltage of inverters enables quick, low-cost reconfigurability without reprogramming ReRAMs, effectively mitigating DNN modeling and SCA vulnerabilities. Our Re^{4}PUF chip has been experimentally validated, achieving a low Bit Error Rate (BER) of 1% at 85°C, a 7.59-fold reduction compared to existing ReRAM-based PUFs. It also demonstrates robust resistance to both DNN modeling attacks (MLP and Transformer) and SCAs, with success rates of approximately 50% and less than 70%, respectively.**

***Index Terms*—ReRAM; PUF; SCA; DNN modeling attack; Reconfigurable.**

I. Introduction

The rapid progress of smart IoT necessitates enhanced security features, such as unique, device-specific fingerprints for key generation [1]. ReRAM-based PUFs uses inevitable variation in fabrication and redox reaction as the entropy source, which is promising for the high integration density, low operating energy consumption, reduced fabrication costs. However, these attributes also pose significant challenges to reliability, particularly in harsh environments. For instance, elevated temperatures of 85°C can result in a substantial BER of up to 11.5% [2]. Moreover, adversaries may model PUFs by collecting Challenge-Response Pairs (CRPs) and employing DNNs to replicate the functionality of the PUF, thereby impersonating the associated identity. This vulnerability is further intensified by SCAs, where attackers can probe ReRAM conductance [3].

ReRAM-based PUFs. In the early stages, ReRAM-based PUFs mimicked CMOS PUFs (e.g., arbiter PUFs [4], [5]). For instance, Ref. [4] leverages the random conductance of memristors to form two paths with different conduction latencies, generating a final 1-bit response through a D Flip-Flop. Subsequent advancements saw researchers organizing

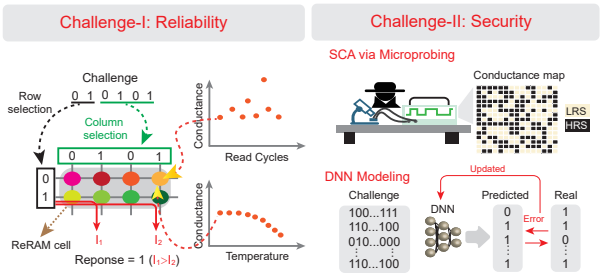

Fig. 1. Challenges faced by ReRAM-based PUFs: (I) Reliability issues arising from read noise and temperature variations. (II) Security vulnerabilities due to DNN modeling attacks and SCAs via microprobe techniques.

ReRAM into 2D and later 3D crossbar structures to serve as memory-based PUFs [2], [6]–[9]. These efforts primarily focused on exploiting the inherent randomness of ReRAM as an entropy source. For instance, Ref. [8] proposes using metal-oxide memristive crossbars to implement PUFs. In their design, the challenge signal serves as both the row and column selection signal, applying voltage to the ReRAM device at the intersection of these signals. The current of the two (even-numbered) columns is then compared to output the corresponding 1-bit 0/1 response.

Challenges. Both arbiter and memory-type ReRAM-based PUFs encounter significant challenges related to reliability and security, as illustrated in Fig. 1.

Regarding reliability, an ideal PUF should produce identical responses upon the same challenge, or a zero BER. However, the reliability of these PUFs is compromised by cycle-to-cycle (i.e., read) variations and sensitivity to temperature changes.

In terms of security, these ReRAM-based PUFs have been validated against various machine learning (ML) and DNN models, including logistic regression, support vector machines, and simple RNNs [2], [6]–[8]. However, their theoretical resistance to ML or DNN modeling attacks is limited due to the strong fitting capabilities of these models [10], which have seen significant advancements in recent years [11].

Current Solutions. To address reliability issues, existing solutions often incorporate additional peripheral circuitry or reprogramming strategies to enhance stability. For instance, Ref. [12] employs post-processing techniques such as Temporal Majority Voting (TMV) and Masking [13], which reduce the BER from 6% to zero. Similarly, Ref. [6] utilizes con-

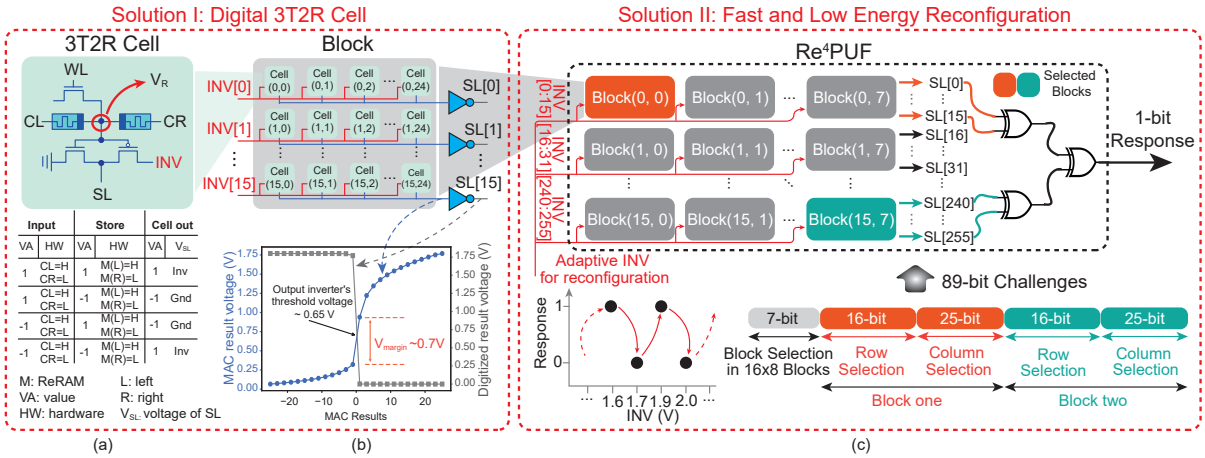

Fig. 2. Re^{4}PUF architecture. (a-b) A novel digital 3T2R cell in a 16×25 block. ReRAM cells form a voltage divider, with output binarized by an inventor. This mitigates reliability issues caused by read noise and temperature variation. (c) Fast and low energy reconfiguration to resist DNN modeling and SCA, by adapting inverter voltage INV in the option set (1.6V, 1.7V, 1.9V, 2.0V)

tinuous programming of each cell for calibration, lowering the BER from 8.4% to approximately 0.014%. However, the inclusion of additional peripheral circuitry undoubtedly introduces extra energy and area overhead, and reprogramming often requires iterative write-verification resulting in significant latency overhead.

To address the security issue, a promising Concealed ReRAM-based PUF [3], [12] has been proposed. This approach restores private conductance information during use and erases it when not in use, thereby mitigating SCAs. However, the repeated erasing and rewriting (i.e., SET and RESET programming of ReRAM) introduces additional latency and energy overhead. Furthermore, ReRAM devices have finite programming endurance [14]. The process of reprogramming also introduces write noise, resulting in inconsistencies with the original PUF. Additionally, the Concealed ReRAM-based PUF does not offer improvements in reliability.

Our Solution. To address these concerns, we propose a ReRAM-based strong PUF that employs an innovative 3T2R cell design. This 3T2R cell uses a voltage division circuit, significantly improves reliability and achieves a 1% BER at 85°C. Our design includes a lightweight reconfiguration mechanism, achieved by adjusting the supply voltage of inverters in the 3T2R cells. This effectively counters powerful DNN modeling attacks and SCAs. Specifically, our innovations are as follows,

- We identify two primary challenges for ReRAM-based PUFs: reliability and security. Additionally, we introduce two novel criteria for comprehensively evaluating the performance of ReRAM-based PUFs: resistance and reconfiguration.
- To address the reliability issue, we propose a new 3T2R cell design utilizing a voltage division circuit (refer to Solution I in Fig. 2 (a-b)). This design significantly reduces the BER, achieving a remarkable 1% BER at extreme temperatures of 85°C, compared to the traditional weighted-current summation (WCS) designs.
- To tackle the security challenge, we propose a reconfiguration method that eliminates hardware programming by adjusting the inverter supply voltage INV (refer to Solution II in Fig. 2 (c)). This lightweight approach effectively counters DNN modeling attacks (MLP and Transformer) and SCAs, with success rates of around 50% and less than 70%, respectively.

The remainder of the paper is organized as follows: Sections II and III present the proposed PUF design and its workflow. Section IV showcases the experimental results. Finally, Section V provides the conclusion.

II. Re^{4}PUF Design

A. Architecture design

Digital 3T2R cell. Existing ReRAM-based PUFs rely on analog memristor circuits to produce responses [2], [3], [6]–[8]. Typically, these systems utilize WCS to compare bitline (BL) currents with either the current generated from another BL or a reference current, thereby generating a response bit. However, the reliability of these systems is compromised by factors such as read noise and operating temperature fluctuations, which lead to undesirable conductance drift and variation.

Inspired by the voltage division cell design in [15], [16], which substantially enhances reliability in the face of memristor variations, we propose an innovative 3T2R cell (Fig. 2 (a)). This cell integrates two ReRAMs in complementary states, an inverter, and a programming transistor. The output of the ReRAM voltage divider (refer to V_R in Fig. 2 (a)) is binarized by the inverter, effectively performing a multiplication operation between the input (refer to CL/CR in Fig. 2 (a)) and the stored conductance. The binary inverter within the 3T2R cell further mitigates fluctuations in ReRAM resistance, such as read noise and temperature variations. The output voltages from all cells are aggregated on a source line (SL), which, after passing through a binary inverter, yields an output of either "0" or "1" (Fig. 2 (b)).

Fast and low energy reconfiguration. Our designed Re^{4}PUF consists of 16×8 blocks. Cells on the same SL share

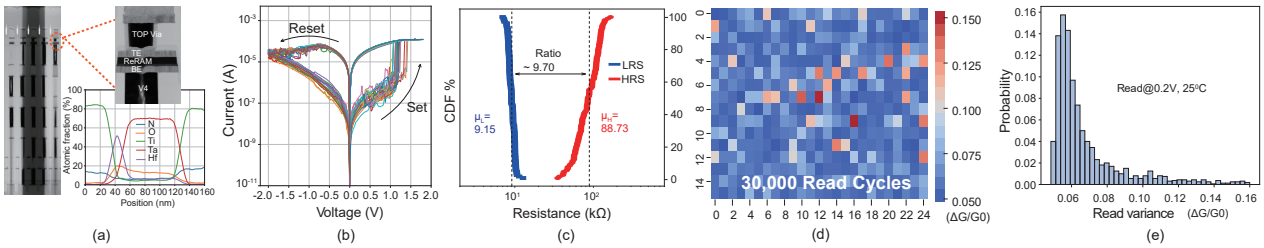

Fig. 3. (a) Cross-sectional Transmission Electron Micrograph (TEM) of a TaOx/HfOx-based ReRAM cell and the Energy-dispersive X-ray Spectroscopy (EDS) compositional profile. (b) DC I-V sweeps over 50 switching cycles. (c) LRS/HRS cumulative resistance distribution for 50 switching cycles. (d-e) The conductance variation heatmap and histogram of all cells within a block after being read 30,000 times at a voltage of 0.2V. ΔG stands for the variance conductance, and G0 is initial conductance.

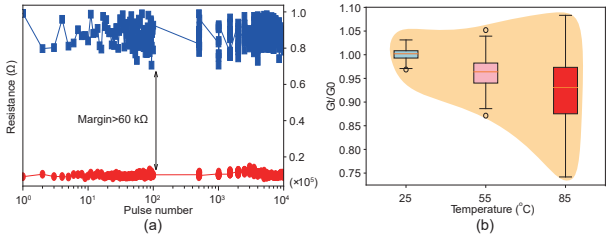

Fig. 4. (a) The endurance of LRS/HRS ReRAM cells. (b) Changes in conductance after baking for 24 hours at 25°C, 55°C and 85°C, Gt represents the measured conductance after baking, G0 is initial conductance.

the same inverter supply voltage INV. Different SLs have different INV voltages. Each combination of INV voltages of all SLs yields a unique PUF. Therefore, adjusting the INV vector reconfigures the PUF (Fig. 2 (c)). Here, each SL opts for one of four INV biasing voltages (1.6V, 1.7V, 1.9V, 2.0V), resulting in a vast selection space of 4^{256} for 256 SLs. An 89-bit challenge is partitioned into three segments: block selection (7-bit, choosing 2 blocks), row selection (32 bits, 16 bits per block), and column selection (50 bits, 25 bits per block). The output signals from the two blocks are individually subjected to two XOR operations, which are combined with an additional XOR operation to generate a 1-bit response (Fig. 2 (c)).

B. Device characteristics

Fabrication. ReRAM cells are integrated via back-end-of line (BEOL) process between Via4 and Metal5 of 180nm technology. They comprise top and bottom TiN electrodes (TE and BE) and a TaO_x/HfO_x switching layer (Fig. 3 (a)).

Entropy source. The operational principle of ReRAM hinges on the reversible switching between a low resistance state (LRS) and a high resistance state (HRS). The transition from HRS to LRS, termed the SET process, and the reverse transition, known as the RESET process, are both driven by the application of voltages exceeding specific thresholds. These processes involve stochastic formation and dissolution of conductive filaments within the memory material. Our experimental 50 cycles I-V sweeps, affirming the reproducibility of this bipolar switching behavior. The threshold voltages for SET and RESET are approximately 1.3V and -1.7V, respectively. Notably, the I-V characteristics for each cycle exhibit variations due to stochastic filament formation/rupture, leading to cycle-to-cycle (i.e., read variance) and device-to-device variation (i.e., write variance) (Fig. 3 (b)).

Write & Read variance. For write variance, HRS and LRS follow Gaussian distributions, with mean values of 9.15 kΩ and 88.73 kΩ respectively. This results in an on/off ratio of 9.7 (Fig. 3 (c)). This discrepancy between the expected and the actual programmed conductance values in both LRS and HRS underscores the intrinsic variability of ReRAM, providing the entropy source.

The read variance of ReRAM cells over 30,000 cycles is illustrated in Fig. 3 (d). The histogram indicates a maximum read variance of 0.16 (Fig. 3 (e)). This read variance can induce instability in PUF responses, presenting substantial challenges to the reliability.

Endurance & Temperature. To investigate the endurance of ReRAM cell, cyclic SET and RESET operations were conducted using a voltage pulse with an amplitude of +1.3/-1.6V and a width of 300/400ns. Simultaneously, a reading voltage pulse with an amplitude of 0.3V and a width of 5ns was applied to the ReRAM cell to measure its resistance. The results (Fig. 4 (a)) demonstrate that ten thousand SET and RESET cycles do not affect the performance of the ReRAM cell.

The conductance variance after baking at 25°C, 55°C, and 85°C for 24 hours exhibits significantly increased variation and a reduced mean value (Fig. 4 (b)). This observation suggests that higher temperatures have a pronounced impact on ReRAM-based PUFs, potentially leading to inconsistent responses and increased BERs.

III. Re^{4}PUF Workflow

The Re^{4}PUF workflow consists of three stages: initialization, reconfiguration, and CRPs generation, as illustrated in Fig. 5 (a).

Initialization. During the initialization phase, the conductance of the 3T2R ReRAM cells are programmed to random binary bits using the intrinsic programming stochasticity of ReRAM. The initialized system is ready for reconfiguration or CRPs generation.

Reconfiguration. Following initialization, the system enters the reconfiguration phase. During reconfiguration, the ReRAMs are not reset; instead, the inverter's voltage INV for each SL is set to specific values (1.6V, 1.7V, 1.9V, and 2.0V), which leads to a unique mapping between challenges and responses.

CRPs Generation. In the CRPs generation stage, the system processes the 89-bit challenge signal. The selection signals

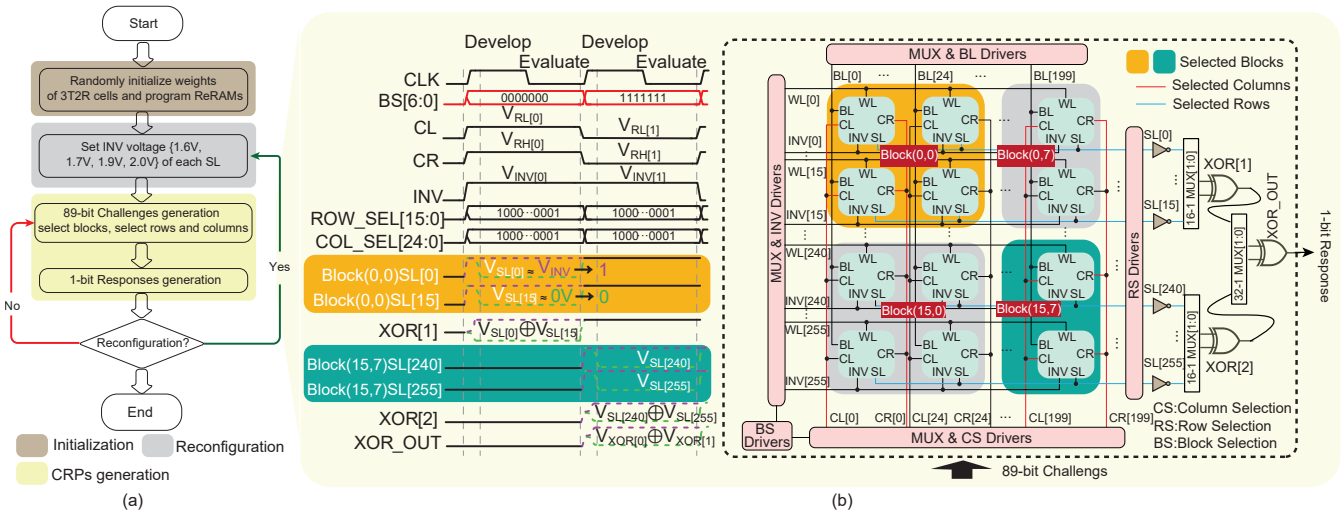

Fig. 5. (a) The workflow includes initialization, reconfiguration and CRPs generation. (b) During the CRPs generation, two blocks are activated by the BS[6:0] signal from the challenge signal. COL_SEL[24:0] signals from the challenge signal is applied to the CL and CR. The chosen block's outputs are selected by ROW_SEL[15:0] signals from the challenge and then undergo XOR operations, which produces the final 1-bit PUF response.

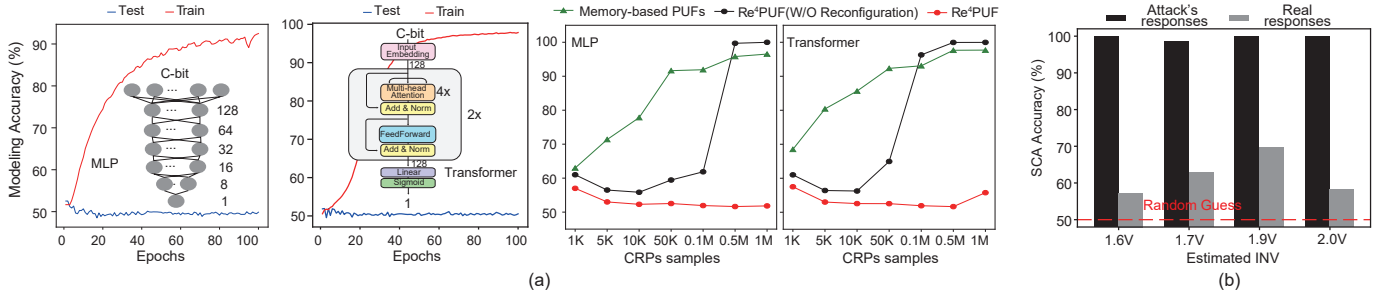

Fig. 6. (a) Re^{4}PUF successfully defends against DNN modeling, including MLP and Transformer. (b) In SCA, attackers fail to predict real responses due to the absence of true INV.

and their interactions during this phase can be further visualized by the waveform shown in Fig. 5 (b), which illustrates how the challenge signals propagate and interact within the ReRAM array. The BS[6:0] of the challenge activates specific blocks within the 16×8 ReRAM block array. The BS[6:3] signals select one of the 16 rows within the block array, while the BS[2:0] signals indicate one of the 8 columns. Following this, the COL_SEL[24:0] signal from the challenge determines which two columns, CL (left column) and CR (right column), within the selected block will be activated. Additionally, the ROW_SEL[15:0] signal from the challenge input refines the selection of row outputs from the activated blocks, further determining which rows will contribute to the response. Once the challenge signals are applied to the selected columns and rows, the activated ReRAM blocks produce two output bits via the two SLs, they undergo XOR operations to produce a unique 1-bit response specific to the applied challenge.

After generating the 1-bit response, the system assesses whether the reconfiguration is required or not.

IV. Experimental Results

The ReRAM of the Re^{4}PUF, featuring a cell size of 0.08μm × 0.08μm and composed of TiN/TaO$_x$/HfO$_x$/TiN layers, was fabricated using standard 0.18μm technology through the following steps: First, the TiN layer was deposited via physical vapor deposition to serve as the bottom electrode. Next, the TaO$_x$ and HfO$_x$ layers were sequentially deposited on top of the TiN layer using atomic layer deposition. Finally, a second TiN layer was deposited as the top electrode, using the same method as for the bottom layer. The electrical characteristics of the ReRAM cell and the performance of the PUF were evaluated using a Keysight B1500A Semiconductor Device Analyzer and a SUMMIT 12000 Probe Station.

A. Resistance

Resistance measures the effectiveness against modeling attacks and SCAs. For modeling attacks, this includes ML models like SVM, as well as powerful DNN models such as MLP and Transformer models [11]. Modeling accuracy ranges from 50% (indicating strong resistance) to 100% (indicating weak resistance). For SCAs, attackers can read ReRAM array conductance using probes [3].

Re^{4}PUF effectively defends against DNN modeling, including MLP and Transformer (refer to Fig. 6 (a) for model details). Increasing training epochs or the number of training samples does not improve the accuracy on test CRPs, which remains around 50%, akin to random guessing. Compared to traditional memory-based PUF in [8] (simulated here for comparison) and non-reconfigurable Re^{4}PUF, the reconfigurable Re^{4}PUF maintains strong resistance (Fig. 6 (a)).

Furthermore, even if attackers read ReRAM array conductance using probes [3] and train DNNs for a given INV

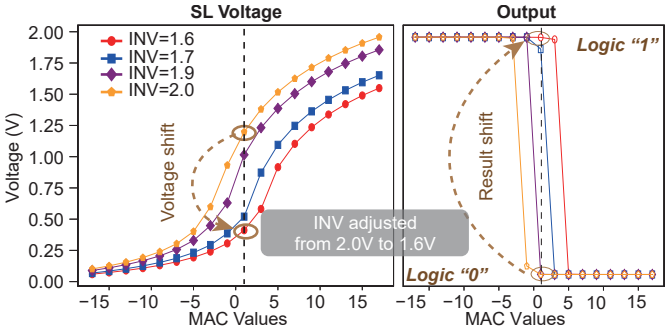

Fig. 7. INV voltage changes SL output, allow fast and low energy reconfigurability. When INV is adjusted from 2.0V to 1.6V, the output voltage of SL shifts from logic "0" to "1".

voltage vector, the real responses remain random guesses due to Re^{4}PUF's reconfigurable nature (Fig. 6 (b)).

B. Reconfiguration

Reconfiguration alters the mapping between challenges and responses, enabling the same input challenge to yield different output responses under different configuration. An optimal reconfiguration strategy should be both energy-efficient and low latency. Concealed ReRAM-based PUF necessitates the erasure of conductances to counter DNN modeling attacks and SCAs, a process that incurs significant costs due to erasing and reprogramming ReRAM cells [3]. To mitigate these drawbacks, our Re^{4}PUF leverages adjustments of inverter supply voltage INV for reconfiguration, avoiding the need to alter cell conductance values.

As illustrated in Fig. 7, modifying the INV voltage directly influences the SL output voltage, thereby changing MAC results. For instance, when the INV voltage is decreased from 2.0V to 1.6V, the SL voltage correspondingly drops from approximately 1.2V to 0.4V. This shift causes the SL inverter's output to transition from logic "0" to logic "1". This reconfiguration method is not only rapid but also get rid of ReRAM programming energy compared to Concealed ReRAM-based PUF [3].

C. Reliability

Reliability is assessed by evaluating the reproducibility of the PUF response under varying environmental conditions, such as temperature fluctuations. This reliability is quantified by the BER. An ideal PUF should produce the same response upon a given challenge across all conditions, or a BER of zero. However, the reliability of ReRAM-based PUFs is often compromised by read noise and conductance drift due to temperature changes.

Traditional memory-type PUFs in [8] (simulated here for comparison) can exhibit a peak BER of 2.5% in repeated CRP generation (Fig. 8 (a)) and up to 30% when more devices are selected for response generation (Fig. 8 (b)). In stark contrast, our Re^{4}PUF, utilizing a voltage divider in each cell, achieves a 0% BER, representing a significant enhancement. As temperature rises, the single ReRAM cell design used in memory-type PUF can lead to conductance changes and current variations, resulting in accumulated errors and higher BER. Our 3T2R

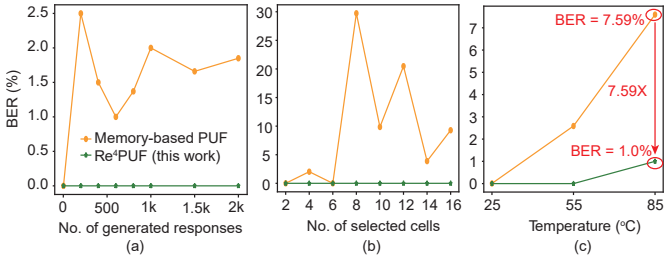

Fig. 8. The BER of Re^{4}PUF under 2k generation cycles (a), the population of selected cells (b), and temperature variations (c).

voltage division design integrates two ReRAMs in complementary states, an inverter, and a programming transistor. This configuration ensures that fluctuations in individual cell conductance do not impact the overall resistance ratio, thereby significantly enhancing reliability. Specifically, while the BER of traditional memory-type PUF in [8] escalates to 7.59% at 85°C, our Re^{4}PUF maintains a low BER of 1% under the same conditions (Fig. 8 (c)).

D. Security Metric

Uniformity (UF) measures the ratio between "0"s to "1"s in PUF's responses. The uniformity of a K-bit-long binary response is defined as the normalized hamming weight [8],

$$UF(B_i) \equiv \frac{1}{K}\sum_{k=1}^{K} b_{ki}, \tag{1}$$

where b_{ki} denotes the k-th bit of the i-th response B_i. The average uniformity is given by,

$$\langle UF \rangle = \frac{1}{C}\sum_{i=1}^{C} UF(B_i), \tag{2}$$

where C is the total number of CRPs. An ideal PUF should yield responses with 50% "0"s and "1"s, as any deviation from this balance could compromise security.

Uniqueness (UQ) measures the dissimilarity between response vectors from different PUF instances under the same input challenge. The uniqueness between two response vectors to the same i-th challenge, from the l-th and p-th PUF instances, is defined as the inter-PUF normalized hamming distance [8], where d measures the hamming distance between two binary vectors,

$$UQ(B_i^p, B_i^l) \equiv \frac{1}{K} d(B_i^p, B_i^l). \tag{3}$$

The uniqueness for the i-th challenge, averaged across all possible pairwise combination of PUF instances, is,

$$\langle UQ(B_i) \rangle \equiv \frac{2}{P(P-1)}\sum_{p=1}^{P}\sum_{l=p+1}^{P} UQ(B_i^p, B_i^l), \tag{4}$$

where P is the total number of PUF instances. Then the final averaged uniqueness is,

$$\langle UQ \rangle \equiv \frac{1}{C}\sum_{i=1}^{C} \langle UQ(B_i) \rangle. \tag{5}$$

The ideal value for uniqueness is 50%, indicating maximum dissimilarity.

TABLE I
COMPARISON OF THIS WORK WITH OTHER RERAM-BASED PUFS.

	VLSI'18 [7]	NE'18 [8]	IEDM'19 [2]	IEDM'20 [6]	Sci.Adv.'22 [3]	DATE'24 [12]	Re^{4}PUF (This Work)
Technology (nm)	200	55	N/A	N/A	130	90	180
NO. of CRPs #	$\binom{M}{m}\binom{N}{n}$	$\binom{M}{m}\binom{N}{n}$	$\binom{M}{m}\binom{N}{n}$	$\binom{M}{m}\binom{N}{n}$	$\binom{M}{m} \times N$	N/A	$\binom{M}{m}\binom{N}{n}$
Working mechanism*	WCS	WCS	WCS	WCS	WCS	VD	VD
Uniformity (%)	50.01	49.5	50.07	N/A	50.13	49.55	48.84
Diffuseness (%)	N/A	50.0	49.93	N/A	49.903	N/A	49.97
Uniqueness (%)	N/A	50.0	49.95	50.004	50.516	49.94	49.99
BER @ 85°C	4.00%	N/A	11.50%	0.01% (reprogramming)	about 4.00%	about 2% (TMV & Masking)	1.00%
Reconfigration	No	No	No	No	SET/RESET	SET/RESET	adaptive INV
Modeling accuracy	0.53	0.5	0.5	about 0.5	about 0.7	N/A	about 0.5
SCA accuracy	N/A	N/A	N/A	N/A	about 0.7	N/A	less than 0.7
Efficiency (pJ/bit)	0.21	0.02	0.004	N/A	12.03	2.225	1.108

* WCS: weighted-current summation. VD: voltage-divider.
# The number of CRPs in each $M \times N$ ReRAM array (block), where m and n represent the selected rows and columns, respectively

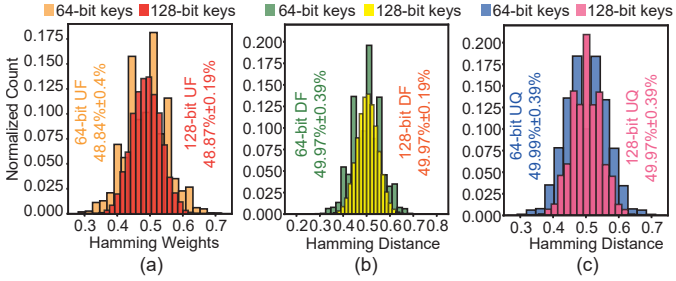

Fig. 9. The UF (a), DF (b) and UQ (c) distributions are computed based on 10k randomly generated 64-bit and 128-bit keys at 0.2V and 25°C.

Diffuseness (DF) quantifies the unique information in a PUF instance. It is defined as the intra-PUF normalized hamming distance d between the i-th and j-th responses [8],

$$DF(B_i, B_j) = \frac{1}{K} d(B_i, B_j). \tag{6}$$

The average diffuseness for all pairwise combination of challenges is,

$$\langle DF \rangle \equiv \frac{2}{C(C-1)} \sum_{i=1}^{C} \sum_{j=i+1}^{C} d(B_i, B_j), \tag{7}$$

with an ideal value of 50%.

The average uniformity, diffuseness, and uniqueness (across 10 dies) of the 64-bit and 128-bit keys generated by Re^{4}PUF, which are composed of 64-bit and 128-bit responses respectively, are close to the ideal value of 50%, as shown in Fig. 9.

E. Comparisons

Table I provides a comparative analysis of the proposed Re^{4}RAM PUF against other state-of-the-art ReRAM-based PUFs. The PUFs discussed in Refs. [2], [3], [6]–[8] utilize challenges to select different ReRAM cells within a 2D or 3D array, performing WCS and current comparison, featuring a large space of CRPs. In contrast, Ref. [12] employs a voltage division scheme with two adjacent device cells, yielding fewer CRPs. To increase the CRP space, we develop 2D ReRAM array using a 3T2R cell design that inherently supports voltage division. The CRP space is calculated as $\binom{M}{m}\binom{N}{n}$ per block.

At 85°C, existing ReRAM PUFs [6], [12] require reprogramming or techniques like TMV and Masking [13] to reduce the BER. In contrast, our 3T2R cell achieves a 1% BER without TMV or Masking.

Current ReRAM-based PUFs typically rely on SET/RESET operations for reconfigurability, whereas we achieve this with simple voltage adjustments. For modeling attacks and SCAs, previous ReRAM-based PUFs are tested against SVM and shallow MLP. We verify our design on DNNs including deep MLP and Transformer models, showing approximately 50% DNN modeling accuracy and and less than 70% SCAs accuracy, which confirms the strong resistance.

V. CONCLUSION

ReRAM-based PUFs have historically faced significant challenges in both reliability and security. Current ReRAM-based PUFs typically operate in a weighted-current summation mode, but due to the inherent noise of the devices, their stability is relatively poor. Furthermore, post-tapeout chips are essentially fixed black boxes, unable to theoretically or practically resist DNN modeling attacks. Existing reconfigurable PUFs based on conductance programming methods require extensive programming operations in practice, which not only incurs significant overhead but also may fail to restore the original PUF functionality. Re^{4}PUF with a 3T2R cell design significantly enhances reliability, while the adjustment of the INV voltage streamlines the reconfiguration process and enhances security. We believe that Re^{4}PUF will provide the hardware security community with a lightweight, secure, and reliable PUF solution.

ACKNOWLEDGMENT

This research is supported by the National Key R&D Program of China (Grant No. 2020AAA0109005), National Natural Science Foundation of China under Grant 62488101, 62495104 and 62374181, Shenzhen Science and Technology Innovation Commission (Grant No. SGDX20220530111405040), Beijing Natural Science Foundation (Grant No. Z210006), Hong Kong Research Grant Council (Grant Nos. 27209621, 17205922, 17212923), Joint Laboratory of Microelectronics (JLFS/E-601/24). This research was partially supported by ACCESS – AI Chip Center for Emerging Smart Systems, sponsored by InnoHK funding, Hong Kong SAR.

REFERENCES

[1] Sudhir Kudva, Mahmut Sinangil, Stephen Tell, Nikola Nedovic, Sanquan Song, Brian Zimmer, and C. Gray. 16.4 high-density and low-power puf designs in 5nm achieving 23× and 39× ber reduction after unstable bit detection and masking. pages 302–304, 02 2024.

[2] MR Mahmoodi, H Nili, Z Fahimi, S Larimian, H Kim, and D Strukov. Ultra-low power physical unclonable function with nonlinear fixed-resistance crossbar circuits. In *2019 IEEE International Electron Devices Meeting (IEDM)*, pages 30–1. IEEE, 2019.

[3] Bin Gao, Bohan Lin, Yachuan Pang, Feng Xu, Yuyao Lu, Yen-Cheng Chiu, Zhengwu Liu, Jianshi Tang, Meng-Fan Chang, He Qian, et al. Concealable physically unclonable function chip with a memristor array. *Science advances*, 8(24):eabn7753, 2022.

[4] Urbi Chatterjee, Rajat Subhra Chakraborty, Jimson Mathew, and Dhiraj K Pradhan. Memristor based arbiter puf: Cryptanalysis threat and its mitigation. In *2016 29th International Conference on VLSI Design and 2016 15th International Conference on Embedded Systems (VLSID)*, pages 535–540. IEEE, 2016.

[5] Rekha Govindaraj, Swaroop Ghosh, and Srinivas Katkoori. Design, analysis and application of embedded resistive ram based strong arbiter puf. *IEEE Transactions on Dependable and Secure Computing*, 17(6):1232–1242, 2018.

[6] Jianguo Yang, Dengyun Lei, Deyang Chen, Jing Li, Haijun Jiang, Qingting Ding, Qing Luo, Xiaoyong Xue, Hangbing Lv, Xiaoyang Zeng, et al. A machine-learning-resistant 3d puf with 8-layer stacking vertical rram and 0.014% bit error rate using in-cell stabilization scheme for iot security applications. In *2020 IEEE International Electron Devices Meeting (IEDM)*, pages 28–6. IEEE, 2020.

[7] Mohammad Reza Mahmoodi, Hussein Nili, and Dmitri B Strukov. Rx-puf: Low power, dense, reliable, and resilient physically unclonable functions based on analog passive rram crossbar arrays. In *2018 IEEE Symposium on VLSI Technology*, pages 99–100. IEEE, 2018.

[8] Hussein Nili, Gina C Adam, Brian Hoskins, Mirko Prezioso, Jeeson Kim, M Reza Mahmoodi, Farnood Merrikh Bayat, Omid Kavehei, and Dmitri B Strukov. Hardware-intrinsic security primitives enabled by analogue state and nonlinear conductance variations in integrated memristors. *Nature Electronics*, 1(3):197–202, 2018.

[9] Yansong Gao, Damith C Ranasinghe, Said F Al-Sarawi, Omid Kavehei, and Derek Abbott. mrpuf: A novel memristive device based physical unclonable function. In *Applied Cryptography and Network Security: 13th International Conference, ACNS 2015, New York, NY, USA, June 2-5, 2015, Revised Selected Papers 13*, pages 595–615. Springer, 2015.

[10] Shaza Zeitouni, Emmanuel Stapf, Hossein Fereidooni, and Ahmad-Reza Sadeghi. On the security of strong memristor-based physically unclonable functions. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2020.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. NIPS'17, page 6000–6010, Red Hook, NY, USA, 2017.

[12] Jiang Li, Yijun Cui, Chenghua Wang, Weiqiang Liu, and Shahar Kvatinsky. A concealable rram physical unclonable function compatible with in-memory computing. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1–6. IEEE, 2024.

[13] Sudhir Satpathy, Sanu K Mathew, Vikram Suresh, Mark A Anders, Himanshu Kaul, Amit Agarwal, Steven K Hsu, Gregory Chen, Ram K Krishnamurthy, and Vivek K De. A 4-fj/b delay-hardened physically unclonable function circuit with selective bit destabilization in 14-nm trigate cmos. *IEEE Journal of Solid-State Circuits*, 52(4):940–949, 2017.

[14] Zhongrui Wang, Saumil Joshi, Sergey E Savel'ev, Hao Jiang, Rivu Midya, Peng Lin, Miao Hu, Ning Ge, John Paul Strachan, Zhiyong Li, et al. Memristors with diffusive dynamics as synaptic emulators for neuromorphic computing. *Nature materials*, 16(1):101–108, 2017.

[15] Miguel Angel Lastras-Montano and Kwang-Ting Cheng. Resistive random-access memory based on ratioed memristors. *Nature Electronics*, 1(8):466–472, 2018.

[16] Qi Liu, Bin Gao, Peng Yao, Dong Wu, Junren Chen, Yachuan Pang, Wenqiang Zhang, Yan Liao, Cheng-Xin Xue, Wei-Hao Chen, Jianshi Tang, Yu Wang, Meng-Fan Chang, He Qian, and Huaqiang Wu. 33.2 a fully integrated analog reram based 78.4tops/w compute-in-memory chip with fully parallel mac computing. In *2020 IEEE International Solid-State Circuits Conference - (ISSCC)*, pages 500–502, 2020.